



**PERIYAR
MANIAMMAI**
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University)
Established Under Sec. 3 of UGC Act, 1956 • NAAC Accredited

think • innovate • transform

PROJECT REPORT

On

PLAYER BEHAVIOR ANALYTICS USING GAME SERVER LOGS

Department of Computer Science and Engineering

Subject: Data Engineering

II Year - B Section

TEAM MEMBERS

S. Amizhthan (125012063272) — Team Lead

N. Magiratchan (1240120121048)

D. Sri Balaji (1240120121070)

H. Mohamed Fasith (1240120121092)

ABSTRACT

Modern multiplayer games generate large volumes of server log data that contain valuable information about player behavior and engagement. However, this raw data is often unstructured and difficult for server administrators to analyze directly. This project presents a data engineering approach to process and analyze game server logs in order to understand player activity patterns and predict potential player churn.

The system collects raw logs generated from a multiplayer game server such as Minecraft and processes them through a structured data pipeline. The pipeline performs data ingestion, cleaning, transformation, and storage to convert unstructured log data into meaningful datasets. Using this processed data, the system analyzes various player behavior metrics such as login frequency, session duration, peak activity hours, rage quit patterns, and overall player engagement.

In addition to behavioral analysis, the project also applies basic predictive techniques to identify players who may stop playing the server, commonly referred to as churn prediction. The results provide valuable insights that can help server administrators improve player retention and optimize server management strategies.

This project demonstrates how data engineering techniques can transform raw log data into actionable insights, showcasing the importance of data pipelines, behavioral analytics, and predictive analysis in modern gaming environments.

TABLE OF CONTENTS

1. Introduction	1
2. Objectives	2
3. System Architecture	3
4. Implementation Steps	4
4.1 Step 1 - Minecraft Server Setup	
4.2 Step 2 - CoreProtect Plugin Installation	
4.3 Step 3 - Understanding Raw Data	
4.4 Step 4 - Data Ingestion	
4.5 Step 5 - Data Cleaning	
4.6 Step 6 - Data Transformation	
4.7 Step 7 - Behavior Analytics	
4.8 Step 8 - Churn Prediction	
4.9 Step 9 - Final Analytics Dataset	
4.10 Step 10 - Power BI Visualization	
5. Player Metrics	16
6. Technologies Used	17
7. Data Engineering Concepts Applied	18
8. Results and Dashboards	19
9. Learnings and Outcomes	20
10. Conclusion	21

1. Introduction

The rapid growth of the online gaming industry has led to an explosion of player-generated data. Multiplayer game servers record every action a player performs — from logging in and chatting to breaking blocks and interacting with the game world. This wealth of data, if processed correctly, can reveal deep insights into player behavior, engagement levels, and retention patterns.

This project, titled Player Behavior Analytics and Churn Prediction Using Game Server Logs, was built as a Data Engineering project for the subject Data Engineering in the Department of Computer Science and Engineering. The project uses a live Minecraft multiplayer server as the data source and implements a full end-to-end data engineering pipeline to collect, clean, transform, and visualize gameplay data.

The project focuses on the following key areas:

- Collecting raw game server logs from a Minecraft server using the CoreProtect plugin
- Ingesting and cleaning the raw data using Python and SQLite
- Transforming raw events into meaningful player metrics
- Applying behavioral analytics to classify player types
- Implementing a basic churn prediction engine
- Visualizing all findings using Microsoft Power BI dashboards

This project simulates real-world data engineering workflows used in top gaming companies to improve player retention and server management.

2. Objectives

The main objectives of this project are:

- To collect raw gameplay logs from a live Minecraft multiplayer server
- To store logs in a structured SQLite database using the CoreProtect plugin
- To ingest and extract relevant datasets from the raw database
- To clean and preprocess the ingested data to make it analytics-ready
- To transform raw events into meaningful player metrics such as session count, chat activity, and block interactions
- To calculate engagement scores for each player using a custom formula
- To analyze player behavior patterns and classify players into types (Highly Active, Active, Casual, Inactive)
- To predict churn risk for players based on their engagement levels
- To build interactive Power BI dashboards to visualize insights
- To demonstrate the application of data engineering concepts in a real-world gaming context

3. System Architecture

The system follows a structured data engineering pipeline, commonly referred to as an ETL (Extract, Transform, Load) pipeline. The architecture is divided into the following layers:

Layer	Component	Technology Used
Data Source	Minecraft Game Server	PaperMC Server
Data Collection	Player Activity Logging	CoreProtect Plugin
Data Storage (Raw)	SQLite Database	CoreProtect DB
Data Ingestion	Extract Raw Tables to CSV	Python, sqlite3, pandas
Data Cleaning	Remove Nulls, Duplicates	Python, pandas
Data Transformation	Create Player Metrics	Python, pandas
Analytics	Behavior Classification	Python
Churn Prediction	Risk Categorization	Python
Visualization	Interactive Dashboards	Microsoft Power BI

The data flows linearly from the Minecraft server through each layer, with each stage enriching and transforming the data further until it reaches the final visualization layer in Power BI.

4. Implementation Steps

4.1 Step 1 - Minecraft Server Setup

The first step was to set up a fully functional multiplayer Minecraft server using PaperMC, a high-performance Minecraft server software. The server was configured with the following settings:

- Multiplayer access enabled for multiple concurrent players
- Server properties configured for optimal gameplay
- Game environment set up for survival gameplay

Players were able to join the server and perform activities such as chatting, breaking and placing blocks, and interacting with the game world. Every one of these activities automatically generated gameplay logs that were captured for analysis.

4.2 Step 2 - CoreProtect Plugin Installation

To capture detailed player activities, the CoreProtect plugin was installed on the Minecraft server. CoreProtect is an industry-standard logging and rollback plugin for Minecraft servers. It automatically records:

- Block break and block place actions
- Player login and logout sessions
- Chat messages and interactions
- Gameplay events and world interactions

All captured data was stored inside a local SQLite database at the path: `plugins/CoreProtect/database.db`

This database became the primary raw data source for the entire project pipeline.

4.3 Step 3 - Understanding Raw Data

Before processing, it was important to understand the structure of the raw data collected from the CoreProtect database. The raw data included:

- Player login and logout timestamps
- Block interaction events (break/place) with coordinates and timestamps
- Player chat messages with timestamps
- Player session records

The raw data was semi-structured and contained multiple tables. It was difficult to analyze directly due to its volume and lack of organization, making data ingestion and cleaning essential next steps.

4.4 Step 4 - Data Ingestion

A Python-based data ingestion system was developed to read and extract data from the CoreProtect SQLite database. The following technologies were used:

- Python — primary programming language
- sqlite3 — to connect and query the SQLite database
- pandas — to manipulate and export datasets

Three separate datasets were extracted and exported into CSV files:

File Name	Description
sessions.csv	Contains player login and logout session records
chat.csv	Contains all player chat messages and timestamps
blocks.csv	Contains all block break and place interaction records

This process successfully converted the raw database tables into structured, usable datasets.

Program:

```
import sqlite3
import pandas as pd
import os

os.makedirs("../datasets", exist_ok=True)
# Path to CoreProtect database
db_path = "../plugins/CoreProtect/database.db"
# Connect SQLite DB
conn = sqlite3.connect(db_path)
# Read session table
sessions = pd.read_sql_query(
    "SELECT * FROM co_session",
    conn
)
```



```

# Read chat table
chat = pd.read_sql_query(
    "SELECT * FROM co_chat",
    conn
)
# Read block activity
blocks = pd.read_sql_query(
    "SELECT * FROM co_block",
    conn
)
# Save CSV files
sessions.to_csv(
    "../datasets/sessions.csv",
    index=False
)
chat.to_csv(
    "../datasets/chat.csv",
    index=False
)
blocks.to_csv(
    "../datasets/blocks.csv",
    index=False
)
print("Data ingestion completed!")
# Close DB
conn.close()

```

4.5 Step 5 - Data Cleaning

The ingested datasets contained various data quality issues that needed to be resolved before analysis. The issues found included:

- Duplicate records from repeated log entries
- Null or missing values in key columns
- Unnecessary columns not relevant to the analysis
- Inconsistent or malformed rows

Data cleaning was performed using Python and pandas, resulting in cleaned datasets:

Cleaned File	Actions Performed
clean_sessions.csv	Removed nulls, duplicates, and irrelevant columns from session data
clean_chat.csv	Filtered valid chat records and removed empty messages
clean_blocks.csv	Removed incomplete block records and standardized event types

After cleaning, the datasets were fully analytics-ready.

Program:

```
import pandas as pd
import os

# Create output folder
os.makedirs("../output", exist_ok=True)

# Load datasets
sessions = pd.read_csv("../datasets/sessions.csv")
chat = pd.read_csv("../datasets/chat.csv")
blocks = pd.read_csv("../datasets/blocks.csv")
print("Original Session Rows:", len(sessions))
print("Original Chat Rows:", len(chat))
print("Original Block Rows:", len(blocks))

# -----
# CLEAN SESSION DATA
# -----

# Remove null rows
sessions = sessions.dropna()

# Remove duplicates
```

```

sessions = sessions.drop_duplicates()
# Keep only important columns
session_columns = [
    "user",
    "time",
    "wid"
]
sessions = sessions[session_columns]
# -----
# CLEAN CHAT DATA
# -----
chat = chat.dropna()
chat = chat.drop_duplicates()
chat_columns = [
    "user",
    "message",
    "time"
]
chat = chat[chat_columns]
# -----
# CLEAN BLOCK DATA
# -----
blocks = blocks.dropna()
blocks = blocks.drop_duplicates()
block_columns = [
    "user",
    "action",
    "type",
    "time"
]
blocks = blocks[block_columns]
# -----
# SAVE CLEANED DATA
# -----
sessions.to_csv(

```

```

    "./output/clean_sessions.csv",
    index=False
)
chat.to_csv(
    "./output/clean_chat.csv",
    index=False
)
blocks.to_csv(
    "./output/clean_blocks.csv",
    index=False
)
print("Data Cleaning Completed!")
print("Clean datasets saved.")

```

4.6 Step 6 - Data Transformation

Data transformation was the core phase of the project. Raw gameplay events were transformed into meaningful player-level metrics using Python and pandas. The following metrics were calculated for each player:

Metric	Description	Formula / Logic
Session Count	Total number of times a player joined the server	Count of login events per player
Chat Count	Total number of messages sent by a player	Count of chat events per player
Block Activity	Total gameplay actions (block breaks + placements)	Count of block events per player
Engagement Score	Overall player engagement rating	$(\text{Session} \times 2) + (\text{Chat} \times 1) + (\text{Block} \times 3)$
Rage Quit Flag	Indicator for very low engagement players	TRUE if session_count < 2

The engagement score formula was designed to give more weight to gameplay actions (block activity) compared to chat and sessions, as they better represent active gameplay. Players who joined fewer than two times were flagged as potential rage quitters.

Program:

```
import pandas as pd
import os
```

```
# Create output folder
os.makedirs("../output", exist_ok=True)

# Load cleaned datasets
sessions = pd.read_csv("../output/clean_sessions.csv")
chat = pd.read_csv("../output/clean_chat.csv")
blocks = pd.read_csv("../output/clean_blocks.csv")

# -----
# PLAYER SESSION COUNT
# -----
session_count = sessions.groupby("user").size().reset_index(name="session_count")
# -----
# CHAT COUNT
# -----
chat_count = chat.groupby("user").size().reset_index(name="chat_count")
# -----
# BLOCK ACTIVITY COUNT
# -----
block_activity = blocks.groupby("user").size().reset_index(name="block_activity")
# -----
# MERGE ALL METRICS
# -----
player_metrics = session_count.merge(
    chat_count,
    on="user",
    how="outer"
)
player_metrics = player_metrics.merge(
    block_activity,
    on="user",
    how="outer"
```

```

)
# Replace NaN with 0
player_metrics = player_metrics.fillna(0)
# -----
# ENGAGEMENT SCORE
# -----
player_metrics["engagement_score"] = (
    player_metrics["session_count"] * 2 +
    player_metrics["chat_count"] * 1 +
    player_metrics["block_activity"] * 3
)
# -----
# RAGE QUIT DETECTION
# -----
player_metrics["rage_quit_flag"] = player_metrics["session_count"] < 2
# -----
# SAVE FINAL DATASET
# -----
player_metrics.to_csv(
    "../output/player_metrics.csv",
    index=False
)
print(player_metrics.head())
print("Transformation completed!")

```

4.7 Step 7 - Behavior Analytics

Using the transformed metrics, a behavioral analytics module was built to identify and classify player behavior patterns. Players were classified into four types based on their engagement scores:

Player Type	Criteria	Description
Highly Active	Very high engagement score	Players who play frequently and are deeply engaged
Active	Above-average engagement score	Regular players who participate consistently
Casual	Below-average engagement score	Occasional players with limited activity
Inactive	Very low or zero engagement score	Players who have barely interacted with the server

The analytics also identified:

- Peak activity time periods when the server was most active
- Low engagement users who may be at risk of leaving
- Rage quit patterns among players who logged in very few times

4.8 Step 8 - Churn Prediction

A basic churn prediction system was implemented to identify players likely to stop playing the server. Churn prediction is a widely used technique in the gaming industry to proactively retain players.

Players were categorized as High Churn Risk if they exhibited the following characteristics:

- Low overall engagement score
- Fewer than average number of sessions
- Reduced block interaction activity
- Rage quit flag set to TRUE

This churn prediction model simulated real-world player retention analysis techniques used in professional gaming companies. By identifying at-risk players early, server administrators can take targeted actions to re-engage them.

4.9 Step 9 - Final Analytics Dataset

After completing all transformation and analytics steps, a final structured dataset was generated and saved as `final_analytics.csv`. This file consolidated all player metrics and analytics results into a single dataset containing:

- Player username / identifier
- Session count
- Chat message count
- Block activity count
- Engagement score
- Player type classification (Highly Active / Active / Casual / Inactive)
- Rage quit indicator (TRUE / FALSE)
- Churn risk level (High / Low)

This final dataset served as the input for the Power BI visualization dashboards.

Program:

```
import pandas as pd
import os

# Create output folder
os.makedirs("../output", exist_ok=True)

# Load transformed data
df = pd.read_csv("../output/player_metrics.csv")

# -----
# PLAYER TYPE CLASSIFICATION
# -----

def classify_player(score):
    if score >= 150:
        return "Highly Active"
    elif score >= 80:
        return "Active"
    elif score >= 30:
        return "Casual"
    else:
        return "Inactive"

df["player_type"] = df["engagement_score"].apply(classify_player)

# CHURN PREDICTION

def predict_churn(row):
    if (
        row["session_count"] < 2 and
        row["engagement_score"] < 20
    ):
        return "High Risk"
    elif row["engagement_score"] < 50:
        return "Medium Risk"
    else:
        return "Low Risk"
```



```

df["churn_risk"] = df.apply(
    predict_churn,
    axis=1
)
# -----
# SAVE FINAL ANALYTICS DATASET
# -----
df.to_csv(
    "../output/final_analytics.csv",
    index=False
)
# -----
# PRINT SUMMARY
# -----
print("\n===== PLAYER ANALYTICS =====\n")
print(df.head())
print("\nMost Active Players:")
print(
    df.sort_values(
        by="engagement_score",
        ascending=False
    )["user", "engagement_score"].head(5)
)
print("\nHigh Churn Risk Players:")
print(
    df[df["churn_risk"] == "High Risk"][
        ["user", "churn_risk"]
    ]
)
print("\nAnalytics Completed!")

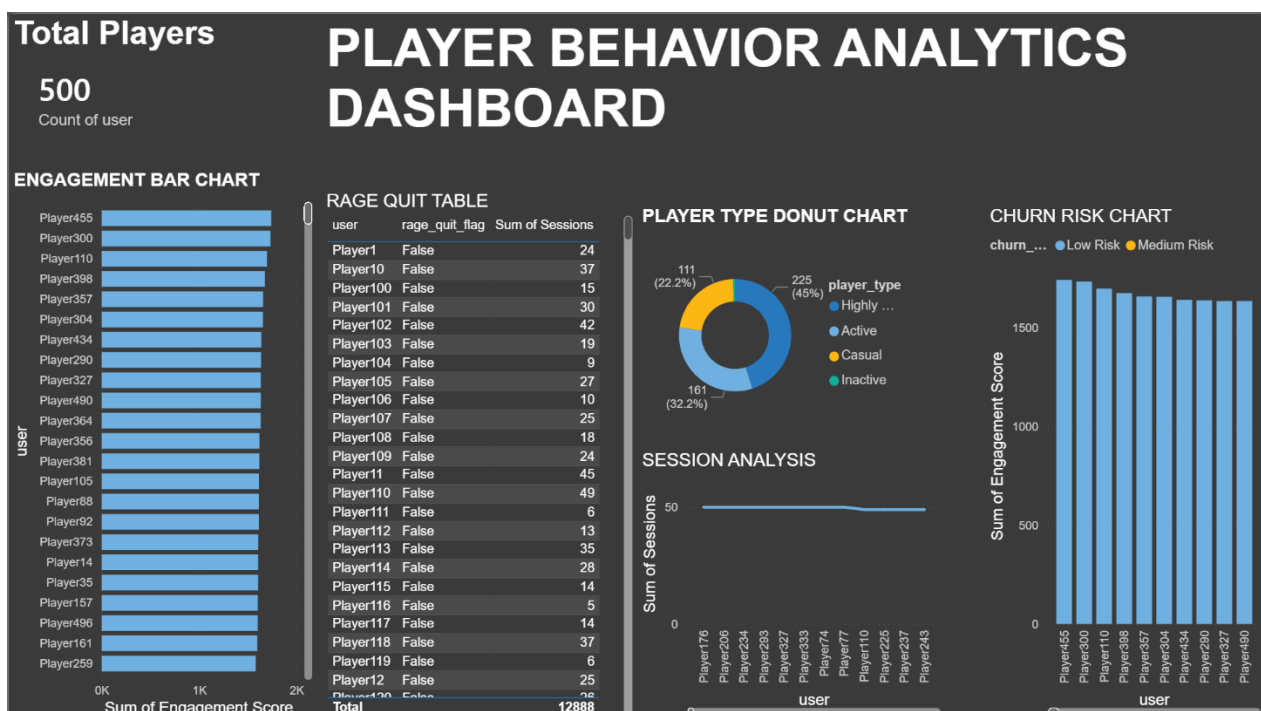
```

4.10 Step 10 - Power BI Visualization

Microsoft Power BI was used to create interactive, visually rich dashboards based on the final analytics dataset. Power BI allowed the data to be presented in an intuitive format that is easy for server administrators and stakeholders to understand.

The following dashboards were created:

Dashboard Name	Description
Player Engagement Dashboard	Displays engagement scores and ranks the most active players on the server
Churn Risk Dashboard	Highlights players categorized as High Churn Risk with supporting metrics
Session Analytics Dashboard	Shows login frequency trends and session patterns over time
Player Distribution Dashboard	Visualizes the breakdown of player types (Highly Active vs Casual vs Inactive)



5. Player Metrics

This chapter describes in detail the player metrics that were created during the data transformation phase of the project.

5.1 Session Count

Session Count measures the total number of times a player logged into the Minecraft server. It is a direct indicator of how frequently a player visits the server. Players with a high session count are considered regular visitors, while players with very few sessions may be at risk of churning.

5.2 Chat Count

Chat Count represents the total number of messages sent by a player in the server chat. Chat activity is an indicator of social engagement. Players who chat frequently are generally more connected to the server community, making them less likely to leave.

5.3 Block Activity

Block Activity records the total number of block-related actions performed by a player, including both block breaking and block placing. This is the strongest indicator of active gameplay, as it shows how much a player is actually engaged in building and exploring within the game world.

5.4 Engagement Score

The Engagement Score is a custom composite metric calculated using the following formula:

$$\text{Engagement Score} = (\text{Session Count} \times 2) + (\text{Chat Count} \times 1) + (\text{Block Activity} \times 3)$$

The weights assigned to each component reflect the relative importance of each activity. Block activity has the highest weight because it reflects actual gameplay. Session count is weighted moderately as it reflects visit frequency. Chat count has the lowest weight as it reflects social activity only.

5.5 Rage Quit Detection

Rage Quit is a gaming term used to describe a player who abruptly stops playing out of frustration after very few sessions. In this project, a player is flagged as a Rage Quitter if their session count is less than 2. This is intended to identify players who tried the server but immediately stopped returning.

5.6 Churn Risk

Churn Risk categorizes players as High Churn Risk or Low Churn Risk based on their engagement metrics. Players with low engagement scores, few sessions, and a rage quit flag are marked as High Churn Risk. This metric is the final output of the churn prediction module.

6. Technologies Used

Technology	Purpose
PaperMC	High-performance Minecraft server software used to host the multiplayer game server
CoreProtect Plugin	Minecraft plugin that automatically logs all player activities into a SQLite database
SQLite	Lightweight relational database used by CoreProtect to store raw gameplay logs
Python 3	Primary programming language used for data ingestion, cleaning, and transformation
sqlite3 (Python)	Python library used to connect and query the CoreProtect SQLite database
pandas	Python data analysis library used for data manipulation, cleaning, and metric calculation
CSV Files	Intermediate file format used to store extracted and cleaned datasets between pipeline stages
Microsoft Power BI	Business intelligence tool used to create interactive visualization dashboards

7. Data Engineering Concepts Applied

This project implemented a wide range of data engineering concepts as part of the pipeline. The table below maps each concept to where it was applied in this project:

Data Engineering Concept	Application in This Project
Data Collection	Raw gameplay logs captured automatically by CoreProtect from the Minecraft server
Data Ingestion	Python script reads the SQLite database and exports data to CSV files
ETL Pipeline	Full Extract-Transform-Load pipeline implemented across Steps 4 through 9
Data Cleaning	Removing null values, duplicates, and irrelevant columns from raw datasets
Data Transformation	Converting raw events into player-level metrics (session, chat, block, engagement score)
Database Storage	SQLite database used as the raw data store; CSV files used as intermediate storage
Behavioral Analytics	Player classification into types based on engagement metrics
Predictive Analytics	Churn prediction based on low engagement indicators
Data Visualization	Interactive Power BI dashboards presenting all key metrics and insights

8. Results and Dashboards

After completing all pipeline stages, the project successfully produced the following outputs:

8.1 Final Analytics Dataset

The `final_analytics.csv` file was generated containing complete player profiles with all computed metrics. This structured dataset serves as a comprehensive record of player behavior across the server.

8.2 Player Engagement Dashboard

The Player Engagement Dashboard in Power BI displays a ranked view of all players based on their engagement scores. Server administrators can immediately identify their top players and compare activity levels across the player base. The dashboard includes bar charts, tables, and filters by player type.

8.3 Churn Risk Dashboard

The Churn Risk Dashboard highlights all players categorized as High Churn Risk. It displays their engagement scores, session counts, and rage quit flags side by side, allowing administrators to identify at-risk players and take action to re-engage them.

8.4 Session Analytics Dashboard

The Session Analytics Dashboard provides a time-based view of player login activity. It shows peak activity periods, total daily sessions, and trends over time, helping administrators optimize server events and maintenance windows.

8.5 Player Distribution Dashboard

The Player Distribution Dashboard shows the proportion of Highly Active, Active, Casual, and Inactive players as pie charts and bar graphs. This provides a quick snapshot of the overall health of the player base.

9. Learnings and Outcomes

Through the successful completion of this project, the team gained practical knowledge and experience in the following areas:

Learning Area	Skills Gained
Multiplayer Server Management	Setting up and configuring a PaperMC Minecraft server with plugins
Gameplay Log Analysis	Understanding the structure and content of CoreProtect logs
SQLite Database Handling	Connecting to and querying SQLite databases using Python
Python Data Processing	Using pandas and sqlite3 for data ingestion, cleaning, and transformation
ETL Workflows	Designing and implementing a full ETL pipeline from raw data to analytics
Player Behavior Analytics	Creating meaningful metrics to classify and understand player behavior
Churn Prediction Concepts	Applying basic predictive techniques to identify at-risk players
Power BI Dashboard Creation	Building interactive business intelligence dashboards for non-technical stakeholders

10. Conclusion

This project successfully demonstrated how raw multiplayer game server logs can be processed using data engineering techniques to generate meaningful behavioral analytics, player engagement insights, and churn predictions for improving player retention and server management.

The project was built end-to-end — from setting up a live Minecraft server to producing interactive Power BI dashboards — showcasing a complete real-world data engineering workflow. Every stage of the ETL pipeline was implemented using industry-standard tools and best practices.

The key outcomes of the project are:

- A fully functional data engineering pipeline was built from scratch
- Raw, semi-structured game logs were successfully transformed into structured, analytics-ready datasets
- Meaningful player metrics were created including session count, chat count, block activity, and engagement score
- Players were classified into behavior types (Highly Active, Active, Casual, Inactive)
- A basic churn prediction engine was implemented to identify at-risk players
- Interactive Power BI dashboards were created to visualize all key insights

This project mirrors the kind of player analytics work done in real gaming companies and demonstrates the power of data engineering in transforming raw log data into actionable business intelligence.

End of Project Report

Periyar Maniammai Institute of Science & Technology

Department of Computer Science and Engineering | Data Engineering | II Year B